Coding with Claude Code

The disciplined engineering loop for computational-materials research: seven phases, the constraint theory underneath them, and one units bug that only a human caught.

Stephen Kerr · Computational Materials Science · Queen's University

TITLE & HOOK · NOT VIBE-CODING · ~5 MIN

Coding with Claude Code, for people who validate the physics

- Not vibe-coding: the disciplined loop that produces code you can trust
- Most 'AI for coding' is written for web apps
- Workflow transfers cleanly; vocabulary doesn't
- You're fluent in the science; the new layer is the engineering workflow
- One running example, two threads, meeting at one number

Who this is for

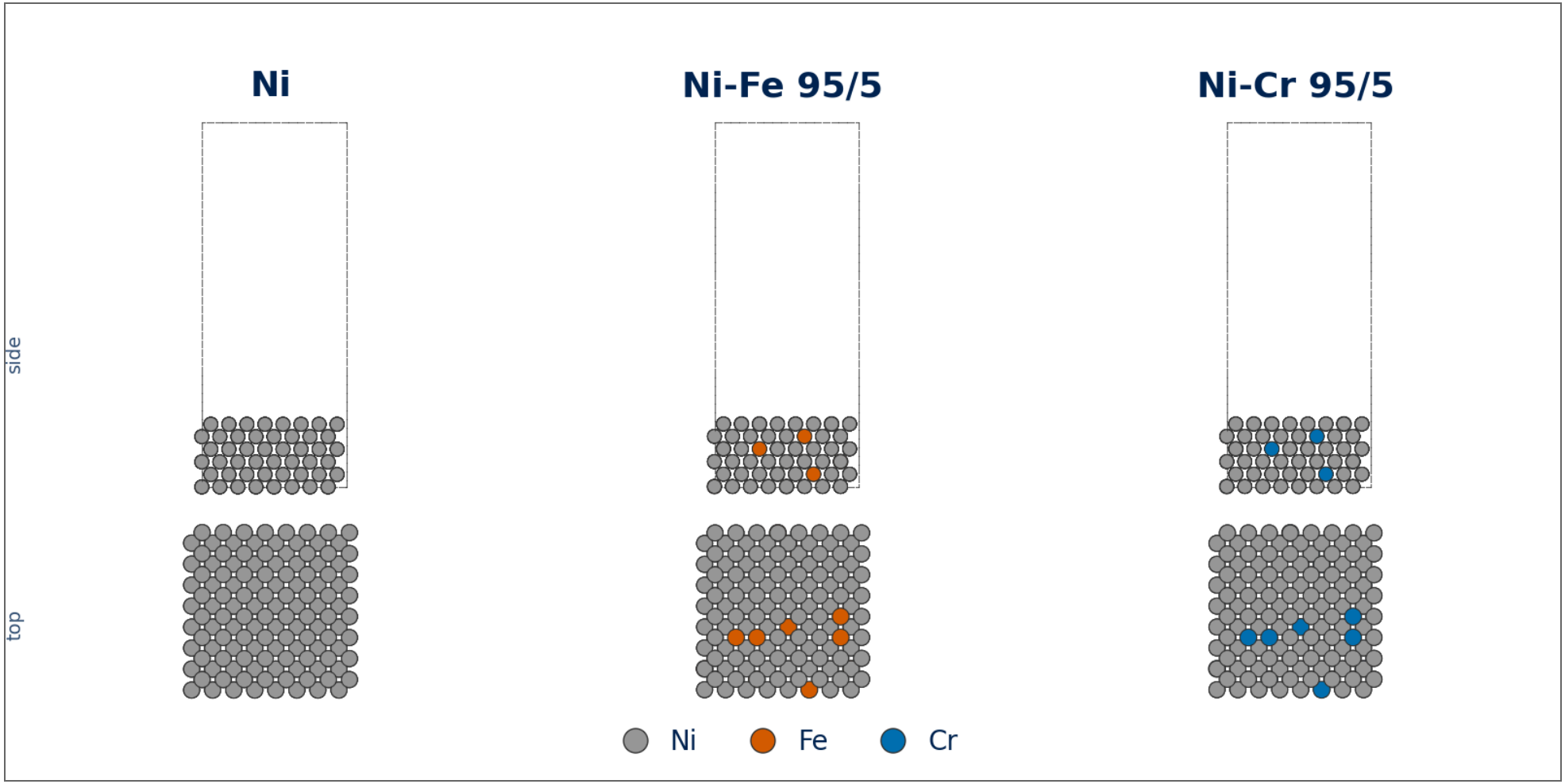
- Comp-materials scientists & grad students who run atomistic/multiscale sims
- KMC, MD/statics, DFT, and write the code that does it
- Given: fluency in the science
- New: the AI-assisted engineering workflow on top
- Explicitly not for vibe coders

The two threads that converge

- Thread 1, substrate: periodic_system_generator → Ni/Ni-Fe/Ni-Cr FCC(001) slabs, vacancies, step edges → LAMMPS+EAM minimize → pyKMC
- Thread 2, correctness: ~23-commit harmonic-TST (Vineyard) prefactor campaign in pyKMC
- They meet at the Ni(100) surface-hop barrier dE
- Same dE the slabs study = same dE that pins the units regression test
- That convergence is why this is THE example

> **ANCHOR** Thread 1 slab generation (Ni/Ni-Fe 95/5/Ni-Cr 95/5 FCC(001)) + Thread 2 HTST prefactor work, meeting at the Ni(100) surface-hop barrier dE (EAM ~0.63 eV, Mehl 1999; DFT ~0.82–0.86 eV, exchange-dominated).

The substrate, built from code: Ni / Ni-Fe / Ni-Cr FCC(001) slabs



THE REFRAME · DEV-SPEAK ⇄ RESEARCH · ~7 MIN

The Rosetta stone: one substitution drives everything

- Deliverable: behaviour a user sees → a physical claim that survives scrutiny
- Every translation is a consequence of that one substitution
- PRD = study/method spec (a mini pre-registration)
- Bug = wrong PHYSICS: no crash, green suite
- QA = validating the physics, not click-testing a UI

The unit of work, translated

- Idea → a physics question, an artifact, or a code capability
- Feature → a new physical capability or measured quantity (the Vineyard backend)
- Bug → wrong physics, often green-and-silent (nu0 in Hz where the clock wants ps^{-1})
- Refactor → restructure the solver without changing the numbers
- Sprint → a simulation campaign (the ~23-commit HTST push)

> `ANCHOR` Idea: 'give pyKMC physically-correct per-event rate prefactors instead of one fixed constant.' Feature: pykmc/rate_constant/backends/htst.py. Bug: nu0 Hz vs ps^{-1}, every rate 10^{12} too large, silently.

Artifacts, agents, and what NOT to carry over

- PRD: spec the END STATE, let implementation move underneath
- research.md asset: a sprint-lived cache allowed to rot
- Memory-less agent: why knowledge lives in contracts, not your head
- Trigger surface: the always-on cost of a skill, minimize it
- Don't import: 'ship fast, user won't notice' (our failure mode is silently-wrong)

> ANCHOR PRD example: 'Deliver D_vac(Ni,500K) with an Arrhenius barrier validated against an EAM-NEB value; archive trajkmc.xyz + reference_table.pickle.' RateConstantConfig is the typed end-state contract.

The dictionary: the unit of work

term	in software development	in materials research
Deliverable	Behaviour a user sees: a shipped, working feature in the product.	A physical claim, or a reusable capability, that survives scrutiny.
Feature	New behaviour a user can see and use.	A new physical capability or a measured quantity (the Vineyard / HTST rate backend).
Bug	Code that crashes or returns visibly wrong output.	Wrong PHYSICS, often with no crash and a green test suite: the silent killer.
Refactor	Restructure code without changing its observable behaviour.	Restructure a solver or driver without changing the numbers it produces.

term	in software development	in materials research
Sprint	A fixed-length iteration delivering a slice of the backlog.	A simulation campaign, or a focused push to land one capability (the ~23-commit HTST push).

The dictionary: plan and board

term	in software development	in materials research
PRD (product requirements document)	The spec of what to build and why: problem, users, acceptance criteria.	A study or method spec, a mini pre-registration: system, conditions, the observable with acceptance bands, output files.
Ticket / Jira card	One scoped, trackable unit of work on a board.	One scoped sub-task of a study or code change ("minimize this slab", "add the units conversion").
Kanban board	Tickets flowing through columns: to-do, doing, done.	The dependency graph of a study: stages in series, seeds / temperatures / compositions in parallel.
Vertical slice / tracer bullet	One thin end-to-end path through every layer, working early.	One end-to-end short run touching every pipeline stage (build → minimize → search → select → analyze).

term	in software development	in materials research
`research.md` asset	A scratch notes file for a task, allowed to go stale.	A throwaway cache of method details (parameters, units, citations) scoped to one campaign, allowed to rot.

The dictionary: the agent and its context

term	in software development	in materials research
Token	The unit an LLM bills and "thinks" in: text encoded to integer ids.	A metered second compute budget; structurally, one catalog entry (a Reference Event index in pyKMC).
Context budget / smart zone / dumb zone	The model's usable working memory; quality degrades as it fills.	Effective context far shorter than the advertised window (~100K usable is a heuristic); the dumb zone manufactures silently-wrong physics.
Compaction vs handoff	Compaction summarizes the chat to continue; handoff starts a fresh session from a doc.	Compaction buys continuity (lossy by omission you don't control); handoff buys purity (a disposable pointer doc for one task).
Memory-less agent	A fresh agent with no memory between tasks; state must be re-read.	Why durable method knowledge lives in contracts and docs, not your head: the

term	in software development	in materials research
		next agent starts blank by design.
Ralph loop / coding-agent execution loop	A dumb bash loop re-running one agent against a prompt and memory file until done (Geoffrey Huntley).	An unattended loop over independent runs (run_overnight.sh under nohup): one run per iteration, append-only memory, per-iteration commit.

The dictionary: quality and design

term	in software development	in materials research
QA / click-testing a UI	Checking the product behaves: click the UI, run the tests.	Validating the physics: barrier vs NEB, detailed balance, MSD linearity, units, one imaginary mode.
TDD / regression test	Write a failing test first; keep tests green to guard behaviour.	A test that pins a PHYSICAL contract and must stay red until the physics is right (a correctness guarantee, not a design driver).
Green and wrong	A passing test suite that does not actually prove correctness.	A passing suite that has ENCODED the physics error and defends it (where the units bug lived).
Deep module / thin interface	A small interface over a large implementation (Ousterhout); the best modules are deep.	The same taste applied to solvers: the seam you wrap a test around, so deep modules lead to strong feedback loops.

term	in software development	in materials research
`Skill / trigger surface`	A reusable, auto-triggering capability; its always-on description is a standing context cost.	Same idea: minimize the always-on description, preferring a dispatched sub-agent or a one-paragraph rule.

FOUNDATIONS · WHY THE WORKFLOW HAS THIS SHAPE · ~9 MIN

The constraints that force the procedure

- Smart zone / dumb zone: effective context is far shorter than the advertised window
- A bigger window doesn't move that limit, it just ships more dumb zone
- Feedback loops are the ceiling, not the model, not the prompt
- Tokens are a metered second compute budget, and mirror the KMC engine
- A procedure you don't understand is one you'll abandon when rushed

Smart zone / dumb zone: a correctness control

- Cost and quality are two facts, not one: the quadratic doesn't cause the degradation
- Cost: attention is $O(n^2)$ in length (compute and memory), so big contexts are expensive
- Quality: positional bias degrades use well before the window fills (Lost in the Middle, TACL 2024; RULER)
- '~100K' is a model-specific heuristic, not a measured constant
- For us the dumb zone manufactures silently-wrong physics: size to the smart zone, clear don't compact

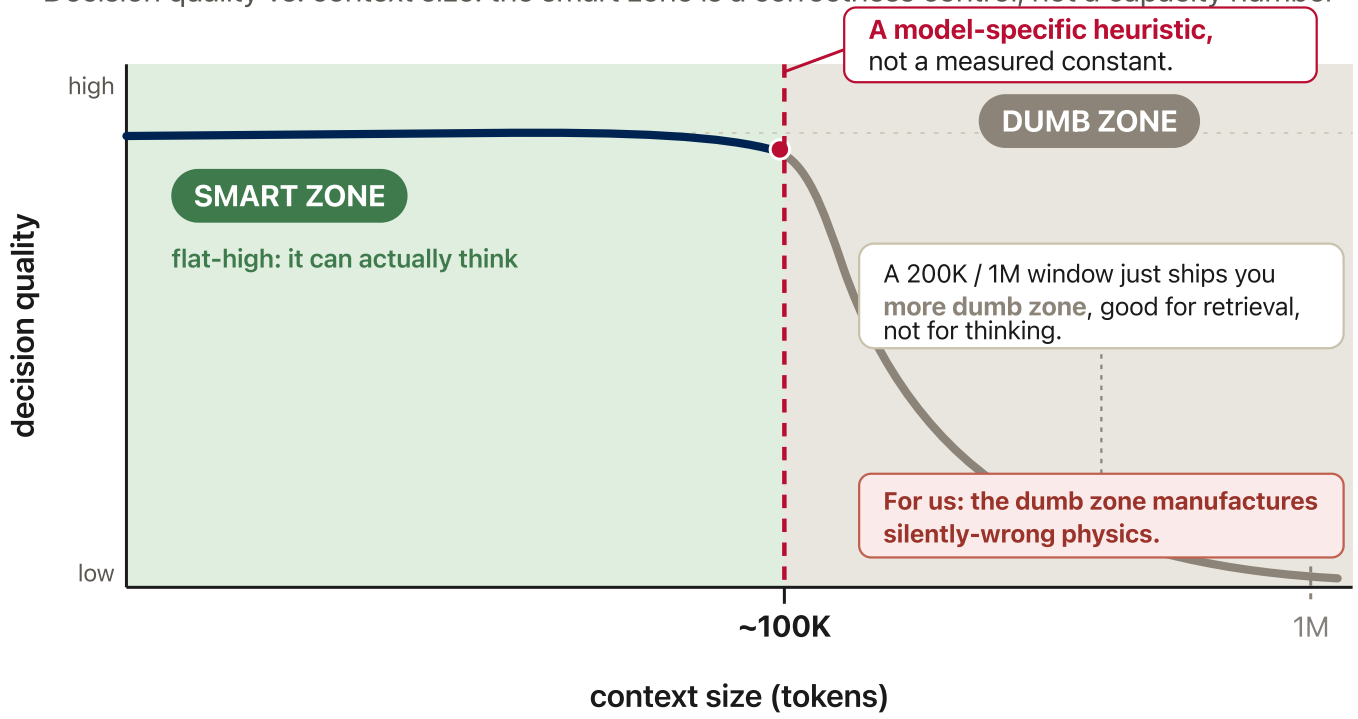
> `ANCHOR` pykmc/htst/constants.py: omega_eV_to_hz() and hz_to_per_ps() are two functions because they are two facts, the fine-grained correctness the agent holds in the smart zone and drops in the dumb zone.

THE PICTURE

Smart zone / dumb zone: a correctness control

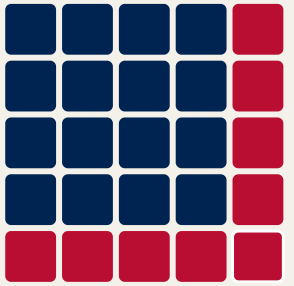
Usable attention degrades before the advertised window

Decision quality vs. context size: the smart zone is a correctness control, not a capacity number



a separate axis: cost

attention is O(n²), not the cause



each token adds a row **and** a column, pairs grow as

$\binom{n}{2} = n(n-1)/2$

Quality dip = positional bias (Liu et al., "Lost in the Middle", TACL 2024; RULER), a separate phenomenon from the O(n²) cost.

Sizing a task to fit the smart zone is a correctness control.
Clear, don't compact: start each unit of work from a fresh, small context.

Feedback loops are the ceiling, and ours can be green-and-wrong

- Web feedback: types, unit tests, lint, a screenshot
- Ours adds a second tier, the physics: detailed balance, NEB match, MSD linearity, one imaginary mode, ~THz prefactor
- 'Green and wrong': a passing suite that has ENCODED the error and defends it
- Deep modules → testable → strong loops; a bad codebase makes a bad agent
- Ousterhout is skeptical of strict TDD: our regression test pins a physical contract, it doesn't 'drive design'

> `ANCHOR` HTST_Test_Report/run_htst_ab.py: paired constant-vs-htst runs on matched seeds, built to fail if HTST yields no finite nu0, a physics-tier feedback loop. RateConstantConfig.require_one_negative_mode encodes physics in the type.

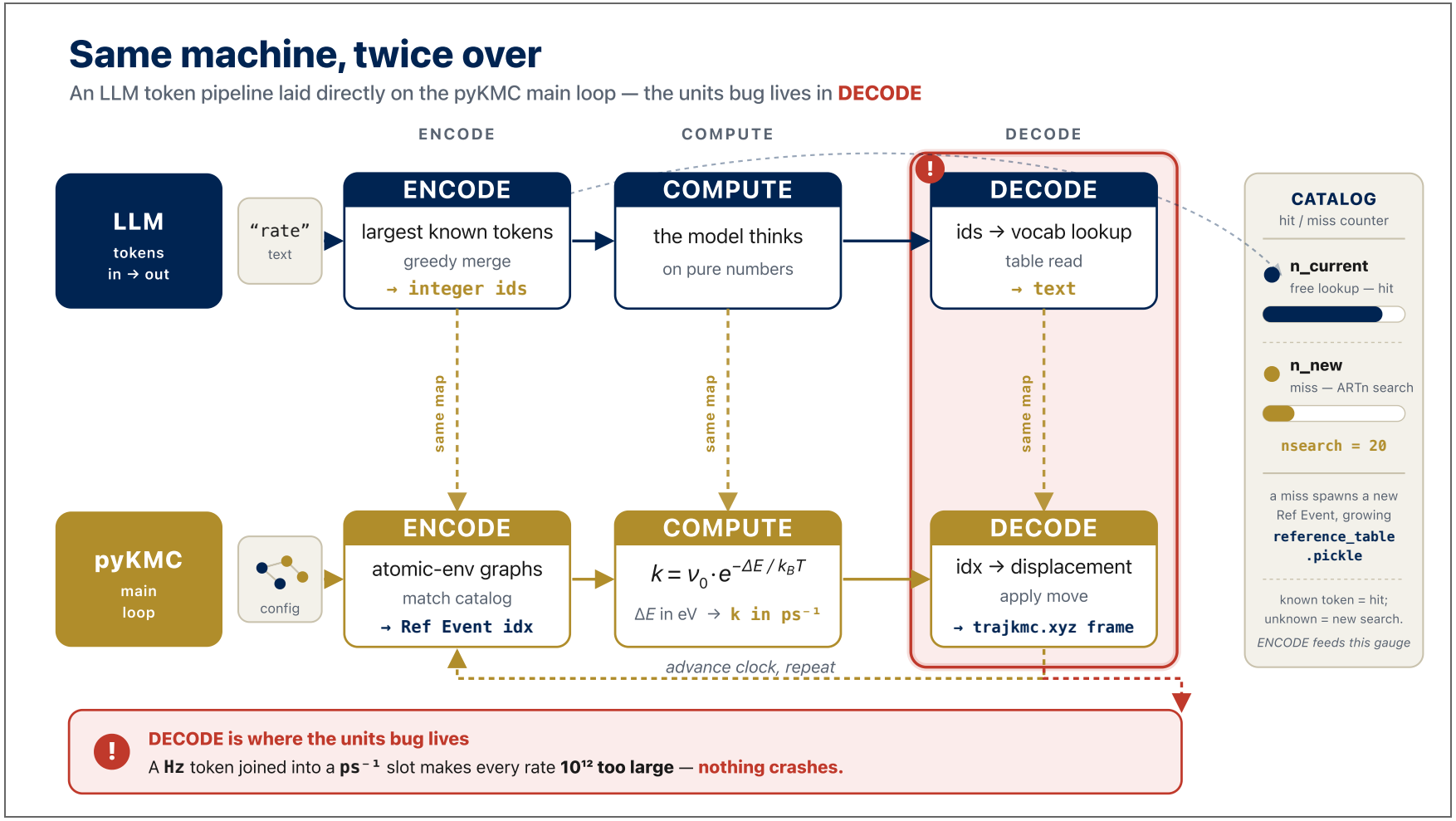
Tokens are the currency, and the KMC engine in miniature

- Tokens: a metered second budget, billed per call, input priced apart from output
- Feed a summary contract plus a file path, never the multi-MB dump
- reference_table.pickle IS a learned vocabulary; the KMC loop IS encode/compute/decode
- n_current = vocabulary hit (free); n_new = miss (nsearch=20 ARTn search)
- Decode silently corrupts when the number carries a different unit than the consumer assumes

> ANCHOR validation_basin: trajkmc.xyz >1MB, reference_table.pickle ~5.7MB, lammps.log.0 ~11MB. The Ref Event is a token id; the Hz→ps^{-1} bug is a decode failure.

THE PICTURE

Tokens are the currency, and the KMC engine in miniature



THE SEVEN PHASES · THE SPINE OF THE METHOD · ~11 MIN

Idea → Research → Prototype → PRD → Kanban → Execution → QA

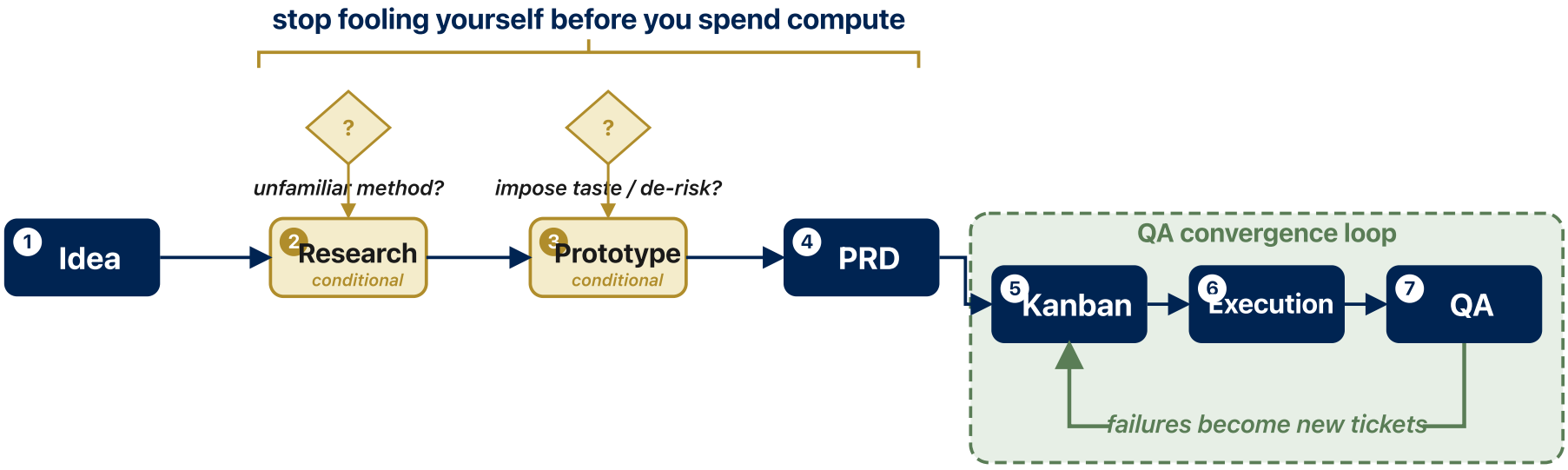
- Two conditional phases: Research (2) and Prototype (3), skipped when small/familiar
- Last three (Kanban→Execution→QA) form a loop until the science converges
- QA failures become new tickets → back to Kanban
- A map, not a gate: tiny work skips 2, 3, most of 4
- Value: names for the steps, so nobody silently drops QA

THE PICTURE

The shape: a linear front half, a converging back-half loop

The seven-phase loop

Linear front half, two conditional phases, and a QA convergence loop



MATERIALS MEANING

1 Idea sharp dissatisfaction or a target number

2 Research cache method details; stamp build / date

3 Prototype sweep knobs till the catalog stabilizes

4 PRD **RateConstantConfig** + **DO NOT MIX** units header

5 Kanban dependency graph; cut vertically

6 Execution build launches real compute (mpirun / srun)

7 QA a human reads units & magnitudes

green tests are not validated physics

ONE LINE PER PHASE

The seven phases, in a simulation campaign

#	Phase	What it means for a study
1	Idea	A physics question, an artifact, or a code capability. State a sharp dissatisfaction or a…
2	Research (conditional)	Cache method details you'll forget: parameter semantics, the units convention, failure…
3	Prototype (conditional)	A throwaway convergence or recipe shakedown: sweep knobs on a tiny cell until the catalog…
4	PRD	A study/method spec, a mini pre-registration: system, conditions, observable + acceptance…
5	Kanban	The dependency graph of the study: stages in series (build→minimize→search→select→analyze)…
6	Execution	Same loop, but 'build' launches real compute (mpirun/srun), genuinely asynchronous. One…
7	QA	Validating the physics. The agent-producible half is real (unit tests, type-check, lint…

Phase 1–3: Idea, Research, Prototype

- Idea: state a sharp dissatisfaction or a target number, not a solution
- 'Rates use a constant prefactor and that's wrong', not 'add a function vineyard()'
- Research (conditional): cache method details into a sprint-lived research.md, stamp with conditions
- Profiling harness, not a notebook: eskm ~2.5–6× faster than Python-FD, agreement 0.014%
- Prototype (conditional): sweep knobs on a tiny cell, commit the winner, delete the losers

> ANCHOR Phase 2: commit d52f541 profiling harness (eskm dynamical_matrix vs Python-FD) → benchmarks/htst_prefactor/RESULTS.md. Phase 3: in.minimize_slab_Ni_*.lmp variants; the NiAlH_jea default failed for Ni-Cr, FeNiCrCoAl-heaweight.setfl won.

Phase 4: PRD, spec the end state, grill the design

- PRD = study/method spec: system, conditions, observable + acceptance bands, output files
- NOT the implementation: over-pinned code rots against fast-moving scripts
- Grill the design: which potential? which seeds? what defines convergence? what's the null result?
- require_one_negative_mode encodes the physics in the type
- Record deferred scope: RpaBackend bare-Vineyard until the Sharia–Henkelman κ lands (deferred, not broken)

> `ANCHOR` RateConstantConfig(style: constant|htst|rpa, free_radius, fd_step, nu0_min/max_THz, require_one_negative_mode). vineyard.py header: 'Units convention (DO NOT MIX) ... nu0 output: Hz (linear, NOT angular)'. Review commit a752002 (Hugo Moison).

THE CODE

Phase 4: PRD, spec the end state, grill the design

```
RateConstantConfig(
    style = "constant" | "htst" | "rpa",  # thin interface literal
    free_radius = ...,
    fd_step = ...,
    nu0_min_THz = ..., nu0_max_THz = ...,
    require_one_negative_mode = True,  # physics invariant AT the interface:
)  # a true first-order saddle has
  # exactly one imaginary mode.
```

Phase 5: Kanban, vertical slices, tickets are worktrees

- Board = dependency graph: stages in series, seeds/temps/compositions in parallel
- First ticket is a tracer bullet: one thin slice through every stage
- Cut vertically (build→minimize→search→select→analyze), never horizontally
- Tickets are literally worktree + branch names
- Independent fixes (freeze-core, units) ride parallel branches

> ANCHOR HTST chain: 9327519 base → Vineyard+config+PHONON install → manager-ops → batched compute → reference backfill → active/refined backfill → recycling skip. Worktrees: pyKMC-htst-work, pyKMC-freezecore-work (fix/freeze-core-pbc), pyKMC-htst-units-work (fix/htst-rate-units).

PHASE 6 ZOOMED IN · THE EXECUTION LOOP · ~7 MIN

Ralph: a dumb bash loop that beats orchestration, and you already own one

- Backstop, one feature per iteration, append-only memory, per-iteration commit
- You already run it: run_overnight.sh under nohup
- 'done' must be runnable physics, not a green test suite
- Backstop is wall-clock + queue depth, not just iterations
- set -e vs set -uo pipefail IS the AFK-vs-attended decision

The 'done' predicate must be runnable physics

- Web: agent flips passes:true on green type-check + tests (defensible there)
- Physics: a green suite tells you the code RAN, never that the physics is RIGHT
- passes = a machine-checkable acceptance band in eV/Å/ps, run by the real solver
- Predicate 1: require_one_negative_mode read off the actual dynamical matrix
- Predicate 2: finite-nu0 A/B gate; coverage<1.0 is a logged WARNING, not a pass

> `ANCHOR` run_htst_ab.py: status='passed' only if the failures list (built from physics observables) is empty. A 511-atom Si smoke run hit 33.3% nu0 coverage, logged as partial, not a benchmark.

THE CODE

The 'done' predicate must be runnable physics

```
# run_htst_ab.py: status == "passed" only if failures is empty
if n_events <= 0:
    failures.append("no reference events")
if "nu0" not in htst_table.columns:
    failures.append("HTST table has no nu0 column")
if n_finite_nu0 <= 0:
    failures.append("HTST produced no finite nu0 values")
if nu0_coverage < 1.0:
    warnings.append(...)  # WARNING, not a failure
```

Memory, sizing, commits, and the role shift

- Append, never update: a memory-less agent reconstructs state from the progress file
- Record the physics state: 'coverage 0.71; three events fell back to k0'
- One feature per iteration: compute_prefactors_batch shards the giant Hessian
- Per-iteration commit = bisectable; the surgical one-line units fix d705932 in its own worktree
- Ralph demotes you from anal-retentive planner to requirements-gatherer

> `ANCHOR` Commit 82e7793 compute_prefactors_batch (batch size chosen by the d52f541 profiling harness); c9a74ad never-recompute-recycled = idempotency; chain ec6aa5f→…→c9a74ad bisectable.

AGENTS · DISPATCH A SPECIALIST · ~7 MIN

When the dumb loop isn't the answer: dispatch a fenced specialist

- Ralph wins the unbounded grind; bounded, owned work wants a specialist
- A sub-agent runs in its own clean context, then returns a compressed verdict
- First the taxonomy: skill vs sub-agent vs MCP vs slash-command vs hook
- The economics: a skill is an always-on tax; a sub-agent costs zero until called
- My real roster: a specialist per subsystem, fenced by what it must NOT touch

REFERENCE · THE LAYERS

Five ways to extend the agent, and what each costs

layer	what it is	when it fires / what it costs
Skill	A reusable, auto-triggering capability (a runbook the model invokes when its description matches).	Always-on: its description rides every session whether used or not. The trigger surface (grilling, tdd).
Sub-agent	A scoped specialist you dispatch for one task, with its own fresh context.	Zero until you call it; then it runs in a separate budget and returns a compressed result (pykmc-htst).
MCP server	A connection exposing external tools or data to the agent.	A standing surface like a skill: its tool schemas load into context while it is connected.
Slash-command	A user-typed shortcut that injects a prompt or procedure on demand.	Zero standing cost: nothing is in the prompt until you type it.
Hook	A deterministic rule the harness fires on an event, with no model judgment.	Costs nothing in context and always fires: use it when a rule

layer	what it is	when it fires / what it costs
		must be guaranteed (post-checkout worktree wiring).

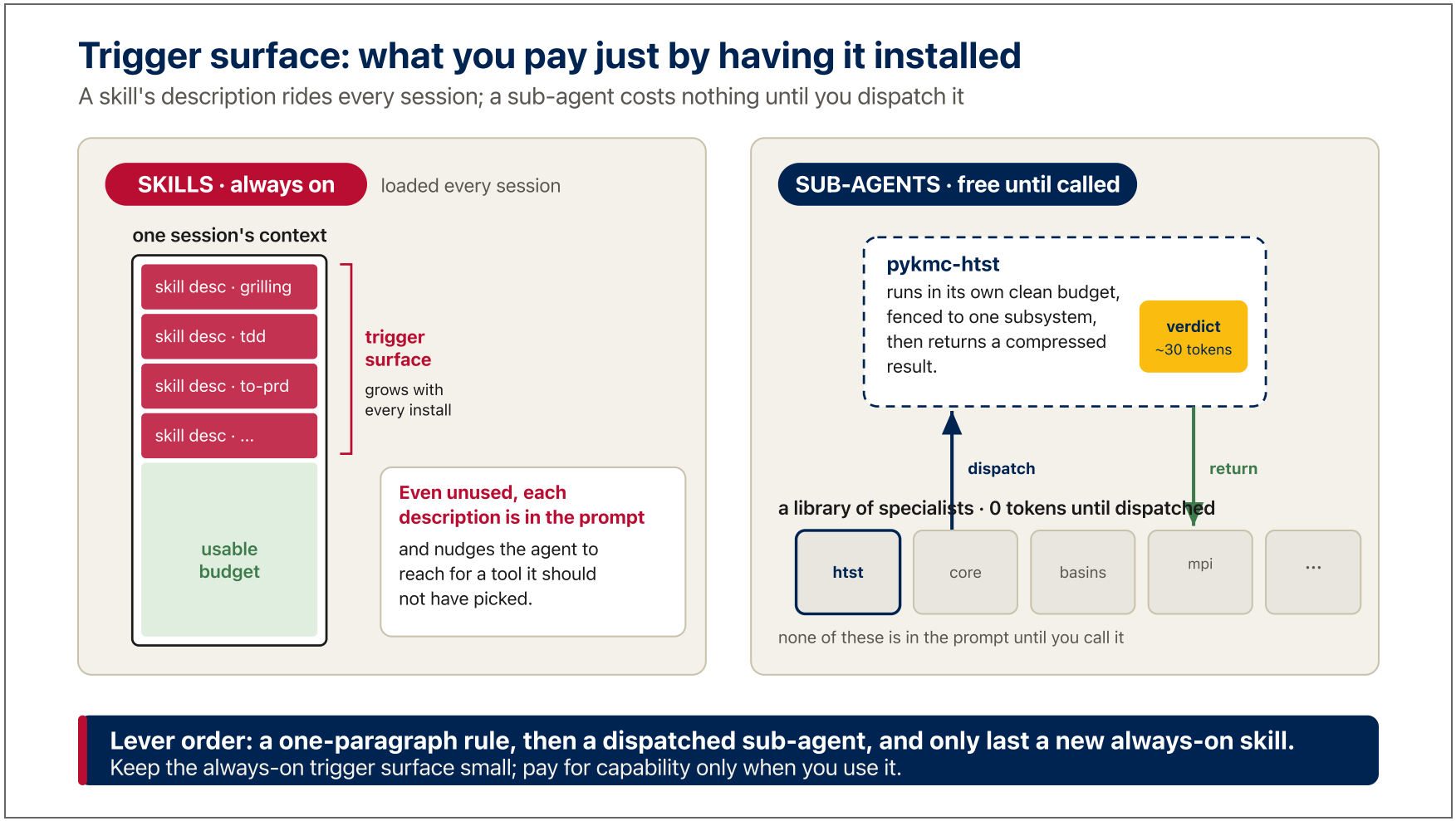
Trigger surface: a skill taxes every session; a sub-agent is free until called

- Every installed skill's description is loaded every session, used or not
- A crowded trigger surface makes the agent reach for tools it shouldn't
- A sub-agent's description costs nothing until you dispatch it
- Dispatched work runs in its own budget, then returns a compressed result
- Lever order: a one-paragraph rule, then a sub-agent, and only last a new always-on skill

> ANCHOR My nine domain agents are kept as dispatched sub-agents, not skills, precisely so their descriptions don't ride every session. The same instinct as 'feed a summary contract plus a file path, never the dump': pay for capability when you use it, not all day.

THE PICTURE

Trigger surface: a skill taxes every session; a sub-agent is free until called



Dispatch by ownership: route by what the work owns, not by which file is named

- One sub-agent per subsystem; its description says what it owns AND what it must NOT touch
- Route by the work's product or domain, not by a filename that happens to appear
- The NOT-clause is the reliability mechanism: a fenced agent stays in its lane
- pyKMC (off-lattice): core / htst / basins / enginemanager-mpi
- pylatkmc (on-lattice): codegen-runtime / ingest-htst-bridge / cross-engine-validation

> ANCHOR A real NOT-clause, from pykmc-core's description: 'Not for HTST Hessian internals, basin escape, or the MPI session pool.' The workspace root carries a domain-ownership table mapping each intent to the one agent that owns it, so dispatch is a lookup, not a guess.

The roster: off-lattice pyKMC, four fenced owners

sub-agent	owns	must NOT touch
pykmc-core	The main KMC loop, environment classification (CNA + nauty hash), the event catalogue + IRA dedup, pARTn search, BKL selection, reconstruction.	HTST Hessian internals, basin escape, the MPI pool.
pykmc-htst	Vineyard attempt frequency ν_0, normal-mode analysis, the Hz→ps^{-1} prefactor boundary, eskm vs FD Hessian.	The main loop, basins, the MPI pool.
pykmc-basins	Super-basin detection, the state-connectivity table, absorbing-Markov exit times (FPTA). Experimental.	Rate internals, codegen.
pykmc-enginemanager-mpi	The pool of parallel LAMMPS instances, MPI rank/communicator splitting, sessions, deadlock diagnosis.	Physics and rate numerics.

ANCHOR A real NOT-clause, from pykmc-core's description: 'Not for HTST Hessian internals, basin escape, or the MPI session pool.' Every row's third column is a fence, not an afterthought: that is what keeps a dispatched agent in its lane.

REFERENCE · THE ROSTER · 2 OF 2

The roster: on-lattice pylatkmc, and the research agents

sub-agent	owns	must NOT touch
engine-codegen-runtime	pylatkmc: the spec→proclist codegen and the C+MPI runtime; Python↔C enum sync.	The catalogue / ν_0, MSD validation.
ingest-htst-bridge	pylatkmc: the curated FCC rate catalogue, HTST prefactors, FAMILY_REGISTRY, ν_0 recovery.	Codegen, the runtime.
cross-engine-validation	pylatkmc: MSD / diffusivity / Arrhenius cross-checks vs pyKMC, the regression gate.	Codegen, catalogue building.

ANCHOR Plus research-output agents kept out of the engine's way: atomsk and periodic_system_generator (structures / slabs), KMC Analysis (notebooks, MSD / basin stats), event_viewer (Dash audit UI), classify_lattice_events, blender_viewer (animations), schematic_viewer (diagrams).

One agent in full: pykmc-htst, and the gun it loads

- Owns the Vineyard attempt frequency ν_0 and the normal-mode analysis
- Load-bearing invariant: ν_0 is produced in Hz, converted to ps^{-1} exactly once, at the backend boundary
- Pinned by a test: tests/htst/test_rate_units_boundary.py (red by 10^{12} if the conversion is dropped)
- Fenced: not the main loop, not basins, not the MPI pool
- This is the agent whose one rule becomes the bug that lands the talk (Phase 7)

> `ANCHOR` pykmc-htst's contract: 'ν_0 produced in Hz (linear frequency); convert to ps^{-1} exactly once at the backend boundary via hz_to_per_ps().' The same boundary the units bug crossed unconverted. The agent's whole job is to defend it.

Set the model on every dispatch; bias up, not down

- Omitting the model silently inherits the session model: a dangerous default
- So set it explicitly on every Task / Agent / Workflow dispatch
- Simplest mechanical work (transcription, one-file edits): Sonnet at effort: max
- Anything with judgment (design, integration, debugging, verification, review): Opus
- Bias up toward Opus: the cost of a wrong number dwarfs the token cost

> `ANCHOR` From the kmc umbrella CLAUDE.md model-selection policy. A dispatched sub-agent inherits nothing from your judgment except what you put in its contract; the model it runs on is part of that contract, so name it.

HONEST LIMIT

The cost is real: skills that fire when they shouldn't

- A skill auto-triggers on description match, even on a 1% guess
- Superpowers' using-superpowers self-invokes aggressively: the chief over-reach
- Symptom: a heavy discipline runs on a routine one-line edit
- Taming lever: an instruction rule in AGENTS.md (instruction files outrank plugin descriptions)
- General rule: prefer a dispatched sub-agent or a one-paragraph rule over a new always-on skill

> **ANCHOR** This is the same trigger-surface cost from earlier, seen from the failure side. The fix is not to uninstall everything, it is to gate the aggressive trigger with a rule the harness reads before the plugin's description.

CONTEXT & HANDOFF · SURVIVING MANY FRESH AGENTS · ~4 MIN

Compaction buys continuity; handoff buys purity

- Grep the convergence line, not the log: 30 tokens vs 30,000
- Compaction is lossy by omission you don't control: drops the load-bearing caveat
- Handoff: a disposable temp-dir pointer doc carrying one task's slice
- Point, not copy: references stay correct as targets evolve
- Portable markdown → adversarial physics review by a different agent

The DIY sub-agent round trip

- Hand off a hard-to-judge bit to a prototype session (separate budget)
- It does the disposable work, returns a compressed verdict
- RESULTS.md IS the handoff-back doc, stamped with its conditions
- Same shape as a dispatched sub-agent, zero machinery
- Promote durable facts to CONTEXT.md; the handoff is the courier, not the archive

> `ANCHOR` Commit d52f541 round trip: the eskm-vs-Python-FD question needed a measurement; benchmarks/htst_prefactor/RESULTS.md returned the verdict (eskm faster, conditions attached) and the planner chose the eskm path.

PRACTICE & HONEST LIMITS · DESCEND OR SKIP · ~8 MIN

What the spine leaves out: clusters, reproducibility, failure modes, objections

- HPC reality: SLURM on the Alliance (DRAC), and the sandbox is the security boundary
- A reproducible artifact, not a reproducible agent (many AI projects don't even run)
- The units bug is one genre: a taxonomy of LLM failure modes
- The four objections you will get, answered
- All four are descend-or-skip depth; the seven-phase spine is the talk

DRILL-DOWN · HPC

Running it for real: SLURM on the Alliance (DRAC)

- Submit and watch: sbatch, salloc, squeue/sq, sacct, scancel
- Always required: --time and --account (plus --mem / --mem-per-cpu, --gpus-per-node)
- $SLURM_TMPDIR is fast node-local scratch, deleted at job end; module load for the stack
- /home (small, backed up) vs /scratch (big, purged) vs /project (shared); Globus for big transfers; one array job, not 1000 sbatch calls
- Security: never an unattended Ralph with --dangerously-skip-permissions on a login node or with live credentials; the sandbox is the boundary

> **ANCHOR** A study run = an array job over seeds/temps, each staging inputs into $SLURM_TMPDIR, launching pyKMC under srun, copying results back to /project before the node is wiped. The Ralph loop runs inside a container/VM, never with live cluster credentials.

DRILL-DOWN · REPRODUCIBILITY

The artifact is reproducible even though the agent is not

- Agent runs are non-deterministic; AI projects often don't even execute (~31.7% fail out of the box)
- The reproducible unit is NOT the agent transcript
- It is: committed code + pinned env (container/lockfile) + fixed RNG seed + the regression test
- Declared vs runtime dependencies drifted ~13.5x in that study: pin them
- Same discipline as any KMC study: fix the seed, archive the inputs, pin the physical contract

> **ANCHOR** Reproducible study = fixed RNG seed for the BKL draws + pinned EAM potential + archived reference_table.pickle + the Ni(100) dE regression test. Rerunnable next year regardless of which agent wrote it. (arXiv:2512.22387)

DRILL-DOWN · FAILURE MODES

The units bug is one genre: a taxonomy

- Slopsquatting: ~19.7% of recommended packages don't exist (USENIX Security 2025); a fabricated import is a supply-chain hole
- Insecure code: ~45% of samples carried an OWASP Top-10 vulnerability (Veracode 2025)
- Higher defect density: AI pull requests ~1.7x the issues of human ones (CodeRabbit 2025)
- Domain silent killers: wrong constants, off-by-one lattice indexing, green-but-wrong tests
- The units bug is the last row; no linter catches it, only a human reading the physics

> **ANCHOR** The Hz to ps^{-1} bug is the 'green-but-wrong test' genre: it ran, the suite was green, the numbers were 10^{12} off. The taxonomy says the units bug was not bad luck, it was a predictable category.

DRILL-DOWN · OBJECTIONS

Objections, answered

- "Won't this deskill students?" AI is the tactical programmer; the human stays the strategic designer, and reviews every diff
- "How do I trust code I didn't write?" You trust the regression tests that pin physical contracts, not the provenance
- "How is a stochastic run reproducible?" It isn't; the pinned artifact is
- "Does this fit open science / FAIR?" Yes: version prompts and CLAUDE.md, disclose AI use, share the artifact
- Disclosure is becoming a norm (Ten Simple Rules for AI-Assisted Coding in Science, arXiv:2510.22254)

DRILL-DOWN · FIGURES AS CODE

Figures are code, not AI images

- AI image generators are unreliable for technical schematics: wrong lattices, missing components, broken connections (SciFlow-Bench)
- One controlled study found 99.8% of generated images contained fabricated detail
- So generate every figure with code: ASE, matplotlib, Graphviz, TikZ, or hand-authored SVG
- The figure becomes a version-controlled, re-runnable artifact: the deck's own thesis
- Every figure here has a source in assets/src or is hand-authored SVG; none is AI-generated

> ANCHOR assets/src/gen_arrhenius_units_bug.py and gen_ni_slabs.py generate the Arrhenius and slab figures (the slab reads the real validation_basin structure); the flow diagrams are hand-authored SVG. Re-run when the physics changes.

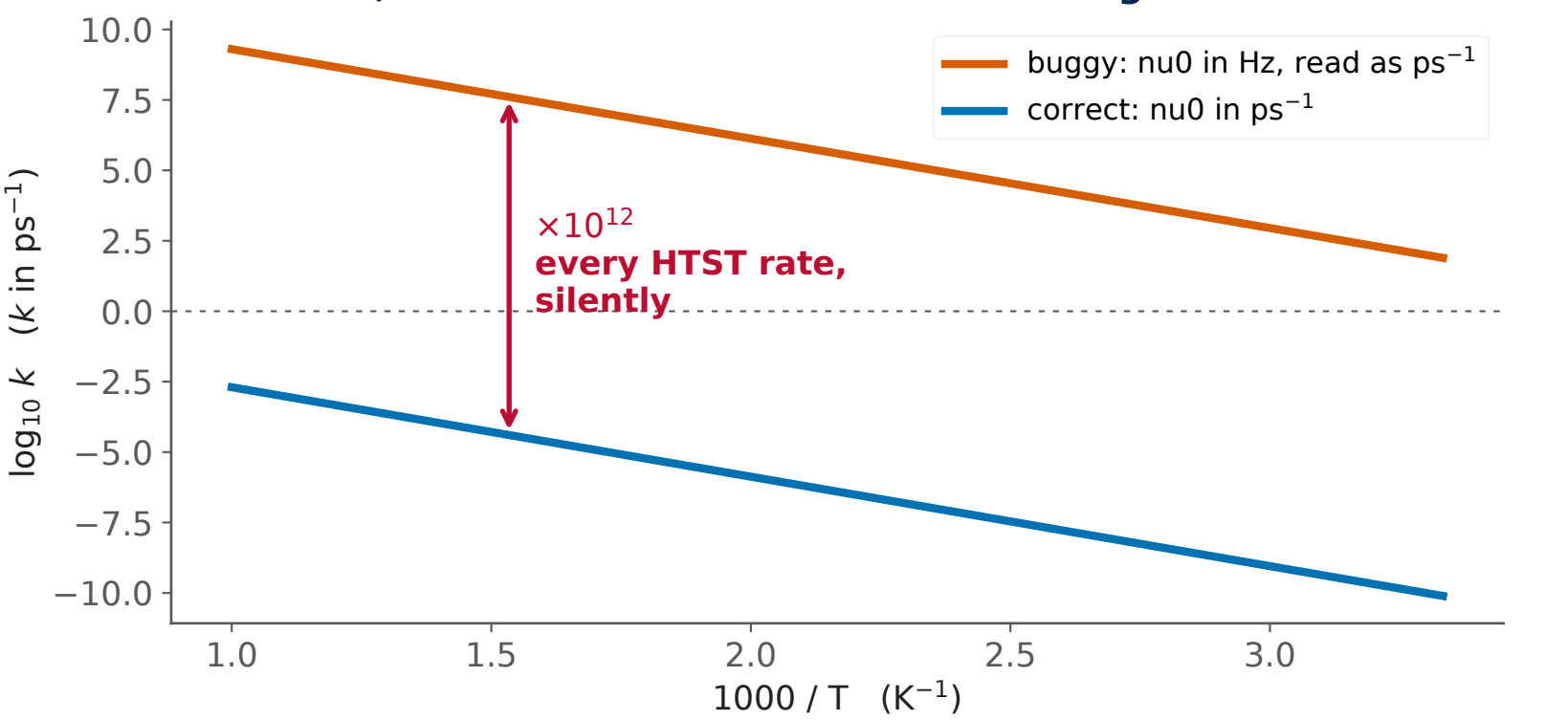
QA, TAKEAWAYS & ATTRIBUTION · ~2 MIN

Phase 7: the units bug, and the one thing to remember

- nu0 emitted in Hz (~1–10 THz), consumed as ps^{-1} → every rate 10^{12} too large
- Nothing crashed; the suite was green; the tests had pinned the error
- Fix d705932: one-line hz_to_per_ps() at the backend boundary
- Regression d6b921f: red by 10^{12} before, green after, at the Ni(100) dE where both threads meet
- QA = validating the physics. Green tests are not validated physics.

THE PICTURE · THE UNITS BUG

The units bug is a 12-decade shift, plotted from the pinned numbers



Ni(100) hop: dE ≈ 0.63 eV (Mehl 1999), nu0 ≈ 3 THz. 1 THz = 1e12 Hz = 1 ps⁻¹. Nothing crashes; the suite is green; τ = 1/k is 1e12 too small.

Attribution & the open questions

- Structure: five Matt Pocock talks; materials mapping is original to this collection
- Documents skills you may already run: grilling, to-prd, to-issues, tdd
- Open: merge Research + Prototype into one 'de-risk the method' phase?
- Open: keep the name 'Ralph' or 'coding-agent execution loop'?
- First cut: the seven-phase mapping is a proposal for discussion, not a settled standard

Speaker notes